THE CHANNEL CHANGED. THE POSTURE DIDN'T.

Google turned off the list upload. We were never uploading lists.

On April 1, 2026, Google disabled Customer Match uploads through the Ads API and replaced them with the Data Manager API — an ingest channel that **requires** per-member consent fields on the wire. For a pipeline that always computed segments server-side and gated them on consent, nothing had to be unlearned. This plan connects Shopify segments to Google along three planes: **Reach** (value-weighted audiences), **Memory** (a BigQuery history), and **Answers** (plain-English questions over both).

For: **Marketing ops, analytics engineering, and the business analyst who owns segments**

Status: **Plan of record · GA4 pipe built · Live Google push routes via the Data Manager API**

Date: **2026-07-10** Depends on: **SEGMENTS-GA4-BIDDING.md · ARCHITECTURE.md · GLOBAL-PAYOUTS.md**

---

0. TL;DR

On **April 1, 2026**, Google disabled Customer Match uploads through the Google Ads API for developer tokens without prior Customer Match history. The replacement — the **Data Manager API** — needs no developer token, authenticates with a plain Google Cloud service account, and fans one ingest endpoint out to Google Ads, Display & Video 360, and GA4. For a pipeline that was always consent-gated and server-computed, this is not a migration. It is the arrival of the channel the architecture assumed.

This plan connects Shopify segments to Google along **three planes**: **Reach** (segments become value-weighted ad audiences), **Memory** (segment, consent,

and revenue history lands in BigQuery), and **Answers** (Google's Conversational Analytics sits on that warehouse so a non-technical stakeholder can ask "which segments drove revenue this week?" in plain English).

One-line recommendation: *Ship the single-US-market pipe now on the Data Manager API, with a `market` key on every row from day one — so going global is adding markets, not rebuilding.*

---

1. What changed at Google — and why it favors this design

RETIRED (APRIL 1, 2026)	REPLACEMENT
Google Ads API `OfflineUserDataJobService` / `UserDataService` uploads	**Data Manager API** — `datamanager.googleapis.com`
Developer token + Ads-specific OAuth plumbing	Standard Google Cloud OAuth; one scope (`auth/datamanager`)
Per-product upload endpoints	One ingest endpoint → Google Ads, DV360, GA4/Firebase, Ad Manager, CM360, SA360
Consent as a policy footnote	Consent fields (`ad_user_data` , `ad_personalization`) **on every member, on the wire**

The Data Manager API also supersedes the GA4 Measurement Protocol for server-side conversion ingestion. The GA4 pipe described in `SEGMENTS-GA4-BIDDING.md` keeps working — this plan adds the direct channel beside it.

The consent posture doesn't change at all: the pipeline computes segments in the data layer, gates them on consent, and hands Google a qualified signal — never a raw list. Google's new API now *requires* what this architecture already *did*.

2. What counts as a segment

Four kinds of group, and they travel differently. Only one ever becomes an advertising audience.

DOMAIN	EXAMPLE	WHERE IT GOES
Peers & Households	VIP repeat buyers; a family sharing one account	The **only** domain projected to Google Ads — consented members, hashed identifiers, value-weighted
Teams	A design team; an approvals group	Insight plane only (warehouse + answers). Never advertising
Organizations	A client company; an agency workspace	Insight plane; any B2B projection is a separate, later decision
Nations	Canada; the EU; Korea	Not an audience — a **boundary**. Scopes campaigns, partitions the warehouse, selects the consent regime and data residency

Rule of thumb: Peers/Households flow *out* to Google; Teams and Organizations flow *up* to insight; Nations decide *where* — and under *which law* — everything runs.

3. The three planes

```
SHOPIFY segment ⟶ WORKER (consent gate · revenue weights) ⟶ DATA MANAGER API ⟶ Ads audi
XANO records    ⟶ nightly delta feed (queued, audited)    ⟶ BIGQUERY warehouse
BIGQUERY        ⟶ Conversational Analytics data agent     ⟶ plain-English answers
```

PLANE	WHAT IT DELIVERS	WHO IT SERVES
Reach	Consent-gated, revenue-weighted Customer Match audiences + value-based conversions for Smart Bidding	Performance media
Memory	Segment / consent / revenue history in BigQuery — atomic nightly loads, every run in the audit chain	Analytics engineering
Answers	Natural-language questions over the warehouse — no SQL, no analyst queue	The BA who owns segments

The Answers plane is the accessibility thesis made literal: connected CRM data that a non-technical teammate can interrogate directly.

---

4. Google-side requirements

One Google Cloud project. Services to enable:

API	WHY
`datamanager.googleapis.com`	Audience ingestion (Customer Match successor) + weighted conversion ingestion
`analyticsadmin.googleapis.com`	Programmatic GA4 audiences mirroring segment definitions
`bigquery.googleapis.com` + `storage.googleapis.com`	The warehouse and its staging bucket
`geminidataanalytics.googleapis.com` + `cloudaicompanion.googleapis.com`	Conversational Analytics data agents (Answers plane)

Authentication: a service account, not user OAuth. Service accounts skip Google's OAuth app-verification queue; the runtime mints short-lived access tokens server-side (JWT-bearer exchange, cached until a minute before expiry).

The credential enters the system through the same supervised key ceremony as every other secret — see `KEY-MANAGEMENT-LIFECYCLE`.

Account linking (human, one sitting): grant the service-account identity access to the destination Google Ads account (requires Ads admin), link GA4 ↔ Ads, and verify **Customer Match eligibility** — it is account-level (policy compliance + payment history) and worth confirming *before* anything is built on top.

Consent on the wire: every ingest carries per-member `ad_user_data` and `ad_personalization`, mapped from the same consent records that already gate the GA4 pipe. No consent, no export — structurally, not procedurally.

Setup is a CLI-and-console sitting; the runtime adds **no new infrastructure dependency** — the existing worker and data layer call Google's REST endpoints directly.

4b. The Merchant calendar — two retirement dates, one checklist

Google is retiring its batch-era surfaces on a published calendar. Both dates share a property worth taking seriously: **nothing errors when you miss them.** A feed simply stops refreshing, listings go stale and are then disqualified — and the reporting keeps rendering the whole time.

DATE	WHAT RETIRES	WHAT REPLACES IT
April 1, 2026	Customer Match list uploads via the Ads API	**Data Manager API** — per-member consent fields required on the wire (§1)
August 18, 2026	**Content API for Shopping**	**Merchant API** — API-native data sources only

After August 18 there is no rollback path: the old surface no longer exists. A cutover that fails on the 19th has no reverse gear — which is why the checklist below ends with a written contingency, not a general assurance.

The migration is coupled. A product feed *reads* the catalog from the commerce platform and *writes* it to Google, and both sides changed: Shopify moved catalog reads to the **GraphQL Admin API**, and Google is moving writes to the **Merchant API**. You cannot push a catalog you can no longer retrieve — so the read side must work before the write side can. (Note the layer distinction: Liquid renders pages; the Admin API is how anything reads the catalog programmatically. "We only use Liquid" is a statement about the theme, not an answer to how a feed gets its data.)

Why the API replaces the spreadsheet. In the Merchant API, a feed is a declared `accounts.dataSources` resource whose `FetchSettings` carry `frequency` (required), `timeOfDay`, and an IANA `timeZone` (UTC by default) — the fetch calendar is part of the API contract, and every sync is timestamped on both sides. A CSV upload has none of that: **you cannot verify a timestamp with a file someone loaded.** The same resource model carries primary, supplemental, inventory, promotion, and product-review data sources — current-state syndication, keyed to the identifiers (GTIN / MPN) your catalog and analytics already share.

The deadline checklist:

1. **Check the data source type on every feed** in Merchant Center — five minutes, no dependencies. Anything still reading "Content API" is on the August 18 clock.
2. **Confirm the read side:** the Shopify GraphQL Admin API is enabled and serving catalog reads today (the REST Admin API is retired; a Liquid theme does not answer this question).
3. **Confirm the write side:** feeds are declared as Merchant API `dataSources` — with `frequency` and `timeZone` set explicitly, not inherited from a file-upload habit.

4. **Key rows to identifiers** (GTIN / MPN) so listings, orders, and analytics join to the same product entity in real time.
5. **Prove the timestamps:** price and availability updates flow through the API with verifiable sync times — the same evidence discipline as an Omnibus price history. If your compliance story depends on *when* data was true, batch files cannot carry it.
6. **Get the contingency in writing before the date, not during it:** how long until a stale feed is detected, what the recovery path is now that rollback does not exist, and who is on call that day. If it proves unnecessary, it cost an afternoon.

For CRM Sync merchants: this platform does not run a Content API feed — product identity publishes through structured data and the API-native rails above, so the retirement is a calendar entry here, not a migration. The checklist exists because most stacks are not in that position.

4c. Two doors into Google — the feed and the SERP

Product presence in Google is two channels off **one identifier**, and a store wins by keeping them consistent, not by choosing between them:

- **The feed → Shopping & free listings.** The Merchant API `dataSources` above push the catalog into Merchant Center; that is what populates Google Shopping, free product listings, and the shopping surfaces AI agents transact against. Keyed on GTIN / MPN.
- **On-page JSON-LD → the organic SERP.** The same product page carries `schema.org/Product` + `Offer` structured data, which is what earns the organic rich result (price, availability, review stars) and lets answer engines cite the product. Keyed on the **same** GTIN / MPN.

When the feed row and the page's JSON-LD disagree — a price in one, a different price in the other — Google distrusts both, and a merchant usually never sees why. The fix is structural: publish product identity from **one record** so the Merchant feed and the page's JSON-LD are two projections of it, not two hand-maintained copies. That is the PIM-plane model — the catalog as a plane, not a page — where the same record ships to Google through the Merchant API *and* renders as JSON-LD on every front-end. See `PIM-ANYWHERE.md` for the projection mechanism and `FRAGMENTS-ANY-FRONTEND.md` for how one record renders identically across Shopify, Webflow, AEM, and plain HTML.

Why this matters for traffic: the readers arriving at this article and the shoppers running Google/Shopify product searches are the same audience one step apart — a question about the mechanism, and a search for the product it produces. Cross-linking the two (the deadline mechanics here → the product surfaces and their structured data) keeps that traffic inside surfaces you own instead of leaking it to a generic result.

4d. JSON-LD: an object at the edge, a row everywhere it is used

The crawl door in 4c depends on structured data actually reaching the crawler. Two things go wrong, and the first is a publishing mistake that produces no error anywhere.

It cannot live in a Rich Text field

Webflow Rich Text stores **sanitised HTML**. Paste JSON-LD into one and it is escaped and wrapped in block elements — a `<script type="application/ld+json">` does not survive. What ships is the JSON **as visible paragraph text**.

The symptom is quiet in every direction. The CMS shows the JSON. The page renders. Nothing logs a failure. And the crawler reads prose, so the product never appears — **not rejected, just never seen.** A reader can also read your markup, which is its own small indignity.

Structured data has to arrive inside a script tag, which means one of three places: an **HTML Embed element**, **page settings custom code**, or **injection at the edge**. The PDP JSON-LD push takes the third route deliberately — generated from the record, not authored in a field, so it cannot drift from the product it describes and cannot be sanitised away by a rich text editor.

What it turns into downstream

Delivered correctly, JSON-LD is a **JSON object** syntactically and an **RDF graph** semantically — `@id` makes nodes referenceable, `@graph` lets several coexist. That form exists for transport and discovery. Both consumers convert it immediately, and in the same direction:

CONSUMER	WHAT IT RECEIVES	WHAT IT BECOMES
Merchant Center (crawl / autofeed)	One `Product` object per PDP	**One row** — `offers.price` becomes the price column, `offers.availability` becomes availability, `gtin` / `mpn` / `brand` become the identifier columns
BQML	Nothing directly. BigQuery can store and query JSON, but a model trains on a **table**	**Feature columns**, extracted and flattened before `CREATE MODEL`, exactly as the pLTV feature SQL does

> **JSON-LD is an object at the edge and a row everywhere it is actually used.**

Which has three consequences worth holding:

- **A missing attribute in the object is a missing column in the row.** There is no partial credit and no error — the offer simply lacks a field Merchant needed.

- **Each consumer flattens differently**, and none of them carries the relationships. Whoever performs the flattening decides what survives.
- **Generate it from the system of record.** Hand-authoring structured data in a CMS field means the object and the row have different parents, and reconciling them later is archaeology rather than a schema question.

The ISO codes are the join, and there is no substitution

Everything in 4d flattens an object into a row. The row's keys are not negotiable, and this is the one place in the estate where the identifier is **not yours to choose**.

FIELD	STANDARD	EXAMPLE
Language	**ISO 639-1**	`en`, `ko`, `ja`
Country / market	**ISO 3166-1 alpha-2**	`US`, `KR`, `JP`
Currency	**ISO 4217**	`USD`, `KRW`
Timestamps	**ISO 8601**, with offset	`2026-09-20T16:08:48Z`

Google will not accept a local taxonomy in these positions. Not a display name, not an internal market key, not a tidier abbreviation. **A feed carrying "Korea" where `KR` belongs is not partially correct; it is rejected or silently mis-targeted.** The same codes scope the review surface — sentiment is partitioned by language and country — and they scope model applicability, because a score fitted on one locale's behaviour does not transfer to another by translation.

And that turns out to be a gift rather than a constraint. Every other identifier in the estate is somebody's private key: a Webflow item ID is Webflow's, a Shopify GID is Shopify's, a row id is the warehouse's, and none of them means anything one system over. The ISO codes are **the only identifiers that mean the same**

thing in every system you run — which makes them the one join that needs no mapping table and cannot drift.

So the rule is short and worth enforcing at ingest rather than at the feed:

- **Store the code, render the name.** `KR` is the value; "Korea" and "한국" are presentations of it.
- **Never key on a display name**, because display names are translated and translations are per-locale — the join would fragment by the very dimension it is supposed to unify.
- **Validate at the boundary.** A two-letter country that is not in ISO 3166 should be refused on write, not discovered in a rejected feed three days later.

5. The warehouse feed

Deltas come off a **sync queue**, not a timestamp watermark — the queue gives idempotency, retry, and dead-lettering, and keeps the feed inside the audit chain (watermarks silently miss hard-deletes).

INGEST OPTION	FIT
GCS batch load + `MERGE` — recommended	Ingestion is free and atomic (a night's load lands completely or not at all); true upserts against the system of record
Legacy streaming `insertAll`	Sub-minute freshness, but billed at a 1 KB-per-row minimum and rows linger in a streaming buffer
Storage Write API	The best engine, but gRPC — reachable from an HTTP-only backend via a thin edge shim, only worth it if a stakeholder needs sub-minute numbers

The loop: read pending → batch to NDJSON → load into staging → `MERGE` on the business key → advance the queue **only on success**. Failures dead-letter with the job ID; every run logs counts into the audit chain.

6. One market or worldwide

Google's fees are near zero either way at this volume — batch loads are free, and the free tiers (10 GiB storage, 1 TiB query per month) cover the working set. The real cost axis is accounts, compliance, and people-time.

	SINGLE US MARKET	GLOBAL ORGANIZATION
Ads structure	One Ads account	Manager account (MCC) + per-market accounts
GA4	One property	Per-market properties, each linked to Ads
Warehouse	One US-region dataset	Region-partitioned datasets (EU data stays in the EU)
Consent regime	US opt-out rules; Consent Mode v2 already exceeds them	EEA: all four Consent Mode v2 signals **mandatory** (built); Korea and Japan add their own consent paperwork
Nations domain	Dormant	Active — where country boundaries do real work
Google cost	≈ $0 / month	≈ $0–10 / month
Real cost	One setup sitting; live in days	Per-market account grants + legal review — people-time, not fees

Recommendation: A with a B-shaped schema. Ship the single US market now; carry a `market` key on every warehouse table and campaign row from day one, and name datasets so regional siblings can appear beside them. Going global then reuses the market plumbing the Canada proof-of-concept already validated.

7. Delivery phases

PHASE	DELIVERS	VISIBLE RESULT
0 — Setup	Cloud project, APIs, service account, Ads + GA4 links, eligibility check	A verified checklist; nothing live yet
1 — Feed	Nightly queued delta load into BigQuery	Row counts + "last updated" any morning
2 — Reach	Live audience ingest via Data Manager API (test mode stays the default)	The segment appears in Google Ads with a count
3 — Freshness	Event-triggered incremental updates (no polling)	Audience counts move on their own after busy days
4 — Mirror	GA4 audiences created programmatically per segment definition	Event-based audiences beside Customer Match
5 — Value	Weighted conversions ingested — the Smart Bidding payoff	Conversion value reflects segment weights
6 — Answers	Conversational Analytics over the warehouse	A plain-English question box for segments

Phase 0 is a supervised human sitting. Phases 1 and 2 are independent and parallel. Phase 6 waits only on Phase 1.

8. The enterprise pivot — when the incumbent stack can't conform

Enterprise organizations holding seven-figure Salesforce/Adobe commitments are waiting for those platforms to conform to the new advertising regime. They

won't — not because the vendors are slow, but because the regime broke the architecture those stacks were built on:

- **Consent Mode v2** demands per-event, per-signal consent stamped at the data layer. Incumbent CDPs store consent as a field on a record — the wrong place architecturally, and no release cycle relocates it.
- **The Customer Match retirement** (April 1, 2026) deleted the upload pattern every enterprise Google connector was built around. The successor channel expects consent fields on every member, on the wire — which presumes the gate above already exists.
- **Agentic buyers never fire the instrumentation.** An agent completes search → cart → checkout with no click, no page view, no pixel. A seven-figure measurement estate is not underperforming on this channel — it is structurally blind to it.

The CDP that syndicated the page view

The clearest instance is the Segment-class CDP. Its founding primitives are the Universal Analytics worldview promoted to infrastructure: `page()` as a first-class API call, identity rooted in a cookie-scoped `anonymousId`, collection defaulting to a client-side library injected into the DOM. Instead of sending the page view to one destination, it sold sending it to a hundred — so it did not escape the page view's obsolescence, it **syndicated** it: when the cookie/client-script architecture broke, the value proposition broke in every destination simultaneously. Server-side sources and audience products moved the transport, not the worldview — the same page/track/identify schema, the same cookie-rooted identity spine, audiences computed inside the toll booth and synced to ad platforms through exactly the upload patterns retired in April 2026. The pricing makes the toll explicit: metering by monthly tracked users bills the merchant per anonymous cookie, including the overwhelming share who never consent, never sign in, and never buy.

This substrate inverts every term of that model. Consent gates the signal at the source instead of filtering it at the destination; identity is the consented login, not a stitched cookie; audiences are computed in the merchant's own data layer and projected outward; and nobody is billed per ghost.

The pivot is not rip-and-replace. The incumbent stack stays as the system of engagement, and keeps receiving its feeds. What changes is the signal path: identity, consent, mandates, and conversions run through the consent-gated substrate this plan describes — stamped at the source, encrypted in transit, agent-addressable — and the estate consumes from a plane that is admissible under the new rules.

Your stack isn't wrong — it's deaf to the new signals. We don't replace it; we give it ears that are legal to use — and ears an average user in the organization can operate. The substrate ships with its own plain-language surfaces: a question box that answers in English (the Answers plane above), a searchable documentation archive written for operators — this article is part of it — and a campaign wizard with approval gates. The people who own segments day-to-day run this in-house. Conformance here is not a statement of work, and nothing waits on a consulting team ten time zones away.

What AEO means here

AEO — answer-engine optimization — is the discipline of being *cited and transacted with* by AI answer engines and their shopping agents, rather than ranked on a results page. It is the successor to keyword SEO: the inputs are machine-readable content, catalog schema, and structured data; the scoreboard is citations (not positions); and the conversion side — an agent completing a cart from an answer — is only measurable server-side, where this substrate lives. (AEO is unrelated to the accessibility/machine-index score used

for CRM Sync UI components — that is an a11y engineering metric, not a marketing discipline.)

The measurement stack, side by side

MODEL	BUILT TO MEASURE	PRIMITIVE	AGENT-COMMERCE IMPACT	CONSENT POSTURE	FE
Semrush-class (visibility)	Public presence: rankings, now AI-answer citations	The keyword position	SERP → AI answers; rank ≠ cited. **Adapts** — stays useful as the scoreboard for "are we cited?"	None needed — public data only	Pe
Segment-class (CDP)	Client-side behavioral events, routed to N destinations	`page()` / `track()` on a cookie `anonymousId`	Agents never load the script; the audience-sync patterns it fed were retired April 2026. **Bypassed**	Filter at the destination, bolted on after the pipes	Pe bi ar cc
Optimizely-class (A/B)	On-page conversion lift	The DOM variant shown to a browser	Agents render no DOM — nothing to variant-test. **Replaced** by edge/server-side flags	Adds its own script + cookie consent surface	Pe pe im
The standard page-based funnel (ad → landing page → pixel → retarget)	Clicks and page views, stitched into modeled attribution	The click	The agent funnel has no click, page, or pixel — the entire chain simply never fires. **Blind**	Degrades at every hop; modeled numbers paper over the gaps	M pe th ab
Server-side, agent-ready, consent-tracked (this plan)	Authenticated requests: tool calls, mandates, transactions — plus human-web events server-side	The consented event	**Native** — the protocol endpoint is the measurement point; the same plane covers browsers and agents	Gates at the source; stamped per event and per member	In pr cc

The value-add of the last row, stated plainly: **deterministic** (a ledger, not a model — every conversion carries its transaction and mandate); **channel-complete** (one plane sees the browser funnel *and* the agent funnel the others cannot); **admissible** (Consent Mode v2 and Data Manager requirements are satisfied by construction, not by retrofit); **auditable** (every number traceable through the audit chain); and **operable in-house** (the surfaces are built for the segment owner, not a retained integration team). The four incumbent rows are not made worthless — the scoreboard keeps score and page-based testing still serves the human web — but every row above the last one is measuring a shrinking share of the funnel, on rented collection, at a per-contact price.

Measurement without a browser

The forward question decides tooling strategy: what happens when measurement must include agents and machines that never open a browser?

Every load-bearing assumption of the current toolchain — a browser implies a person, a session implies attention, a cookie implies continuity — fails at once. A browser emits *behavior* that tools sample, model, and stitch into inferred identity. An agent emits *authenticated requests*: tool calls carrying tokens, mandates, and consent claims. There is nothing left to infer — the call announces who it is, on whose behalf it acts, what it is permitted to do, and what it did. Measurement collapses from probabilistic reconstruction into a **deterministic ledger**. The statistical apparatus of the tag era — sampling, modeled attribution, view-through windows — existed to compensate for not knowing. On this channel, you know.

The observation point moves to the only place the merchant controls: the protocol surface where `discover`, `search`, cart, and checkout calls land. No agent will ever execute your JavaScript. Structurally this is a return to server-log analytics — except the requests are signed, structured, and intent-rich, and the

event that *authorizes* an action is the *record* of it: the permissions bus and the measurement bus become the same bus.

The questions change shape with it. Not "which page converted," but: which agent (attestation replaces user-agent strings — bot management inverts from blocking machines to admitting the right ones); on whose behalf (the mandate chain); with what permission at call time; and which *answer* converted — the funnel now runs indexed → cited → discovered → completed.

Measurement vendors face three doors: adapt what they watch (the visibility scoreboards), insert themselves into the server path (a new toll booth that substrate-owning merchants have no reason to admit), or read from the merchant's ledger — becoming reporting layers over a warehouse they no longer collect for. Collection was the moat; on this channel the merchant owns collection by default, because the protocol endpoint is theirs. Google already conceded the direction — Measurement Protocol and the Data Manager API are server-side ingestion of merchant-owned truth, not tags.

The browser era measured what strangers did on your pages. The agent era notarizes what authenticated parties did with your permission. Agent analytics is not a product to buy; it is a report over data this substrate already keeps.

---

9. Related documents

- `SEGMENTS-GA4-BIDDING.md` — the built GA4 pipe: consent gate, revenue weights, user properties, audiences. This plan's Reach plane is its direct-channel sibling.
- `ARCHITECTURE.md` — where the worker, data layer, and consent plane sit.
- `KEY-MANAGEMENT-LIFECYCLE` — the ceremony that admits the service-account credential.

- `GLOBAL-PAYOUTS.md` — the settlement side of the same market boundaries the Nations domain scopes.
- `PIM-ANYWHERE.md` — the catalog as a plane: one record → Merchant API feed **and** on-page JSON-LD, so Shopping listings and organic SERP rich results never disagree (§4c).
- `FRAGMENTS-ANY-FRONTEND.md` — how that one record renders identically across Shopify, Webflow, AEM, and plain HTML.
- `SHOPIFY-2026-RISK-BRIEF.md` — the August 18 Content API retirement and the coupled GraphQL read-side in risk-brief form (§4b).
